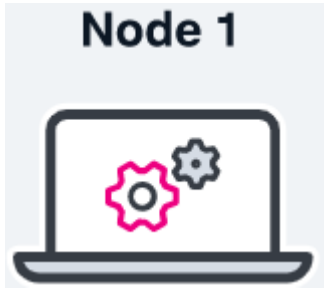
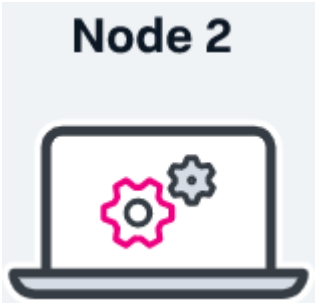
Group Membership and View Synchronous Communication

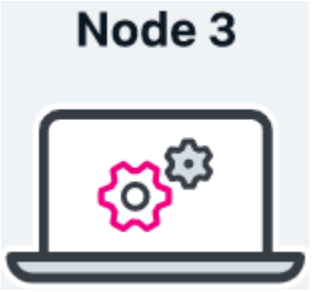
Mohsen Lesani

Group Membership



Who is there?

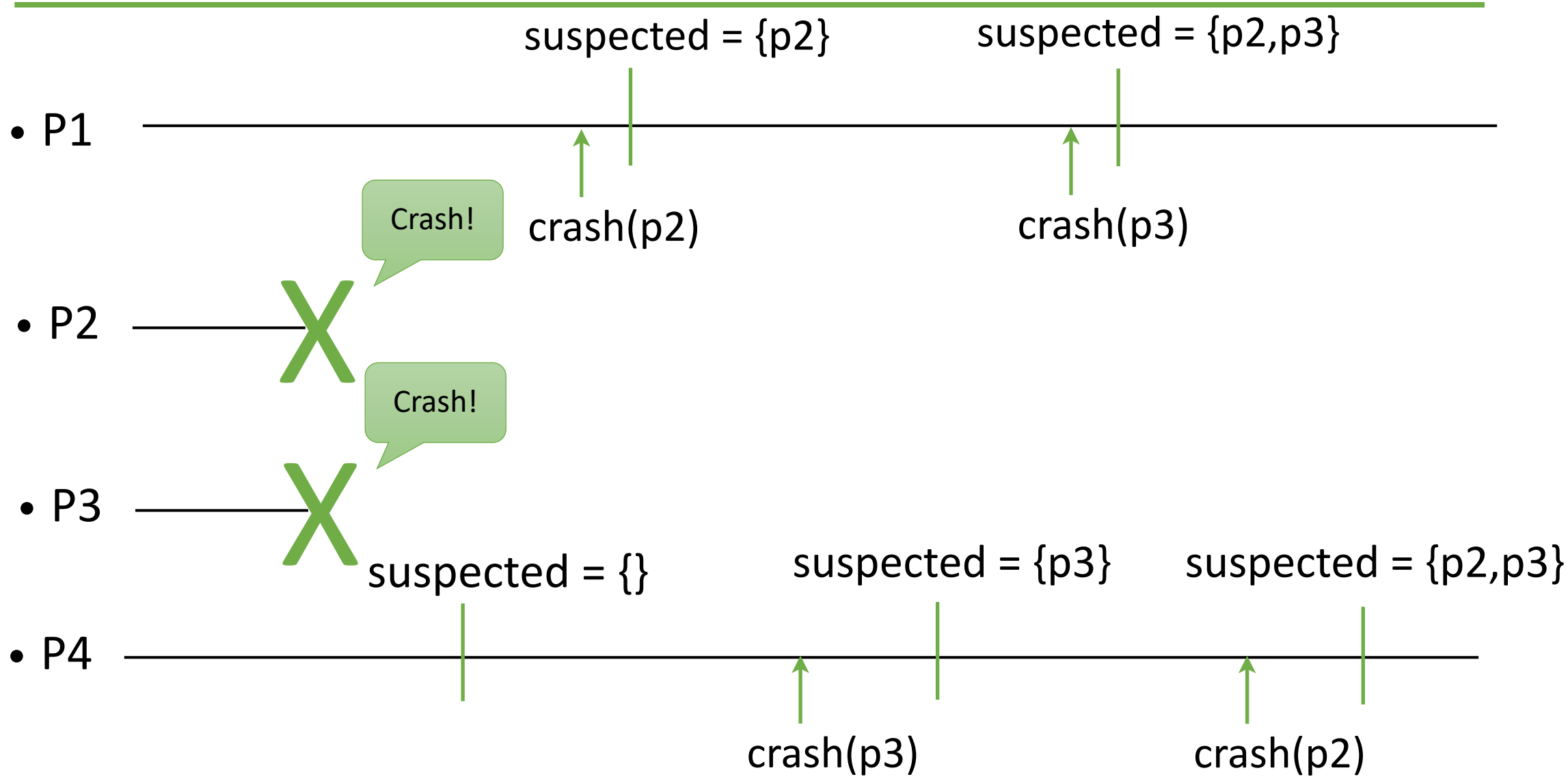




Group Membership

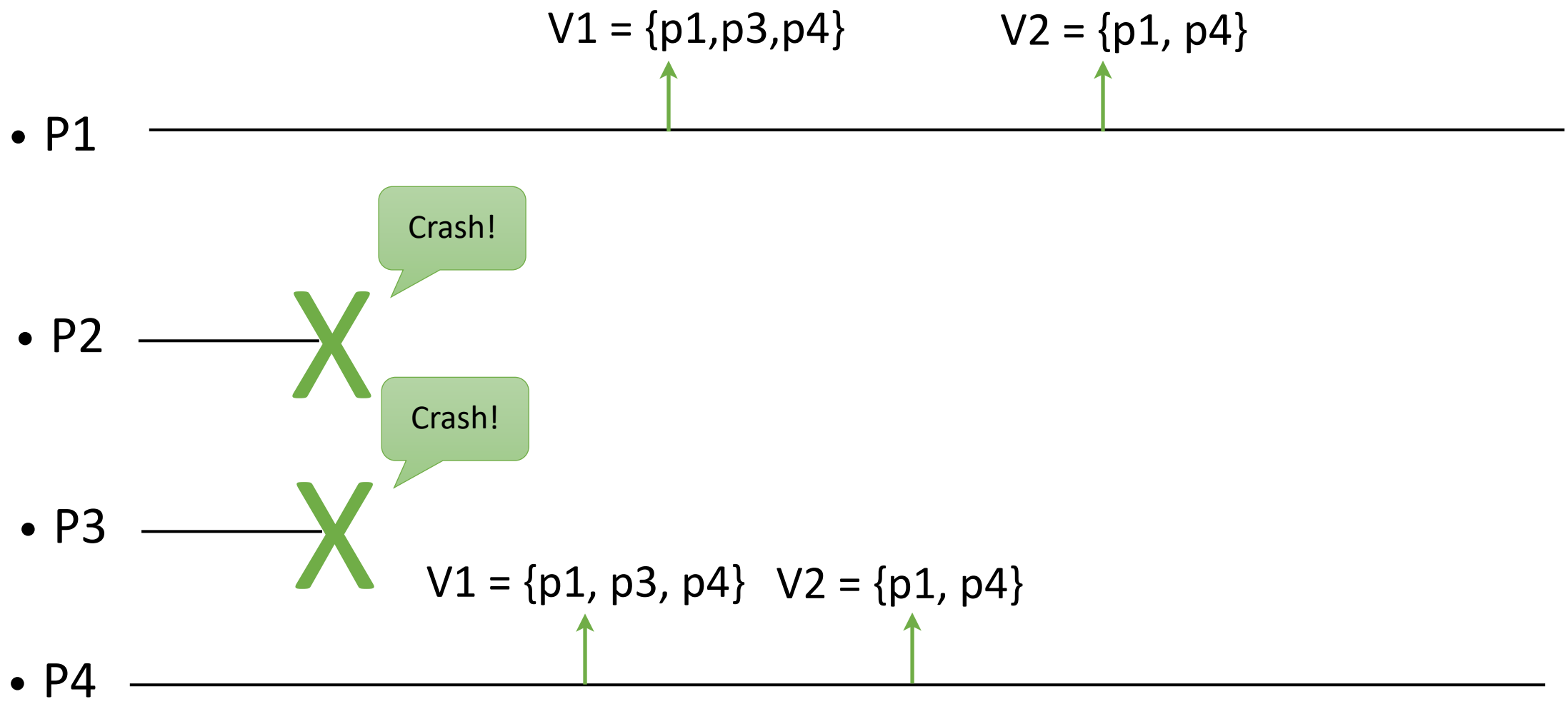
- In some distributed applications, processes need to know which processes are **participating** in the computation and which are not.
- Failure detectors provide such information; however, that information is **not coordinated** even if the failure detector is perfect.

Perfect Failure Detector



Processes do not agree on the set of suspected processes.

Group Membership



Group Membership

- To illustrate the concept, we focus here on a group membership abstraction to coordinate the information about **crashes**
- In general, a group membership abstraction can also typically be used to coordinate the processes **joining** and **leaving** explicitly(i.e., without crashes)

Group Membership

- **Like** a failure detector, the processes are informed about failures; we say that the processes **install views.**
- **Like** a perfect failure detector, the processes have accurate knowledge about failures.
- **Unlike** a perfect failure detector, the information about failures are **coordinated**: the processes install the same sequence of views.

Group Membership

Events

- Indication: < view(V) >

Properties:

- **GM1, GM2, GM3, GM4**

Group Membership (GM)

- GM1. **Monotonicity:** If a process installs view (j,M) after installing (i,N), then $j > i$ and $M \subseteq N$.
- GM2. **Uniform Agreement:** No two processes install views (i,M) and (i,M’) such that $M \neq M'$.
- GM3. **Completeness:** If a process p crashes, then every correct process eventually installs a view (i,M) such that p is not in M.
- GM4. **Accuracy:** If some process installs a view (i,M) and p is not in M, then p has crashed.

Completeness and accuracy are similar to completeness and strong accuracy of PFD.

GM Algorithm

Idea:

Use consensus rounds to install new views

GM Algorithm

Implements: GroupMembership (gmp).

Uses:

PerfectFailureDetector (P).

UniformConsensus (UCons) a sequence.

upon event < Init > **do**

(id, M) := (0, Π)

correct := Π

wait := false

trigger < view (id, M) >

upon event < crash, pi > **do**

correct := correct \ {pi}

wait is true when a view is proposed to a consensus and the decision is not made yet.

GM Algorithm

upon event (correct $\subsetneq$ M) and (wait = false) **do**

id := id + 1

wait := true

initialize uc[id]

trigger <uc[id], propose (correct)>

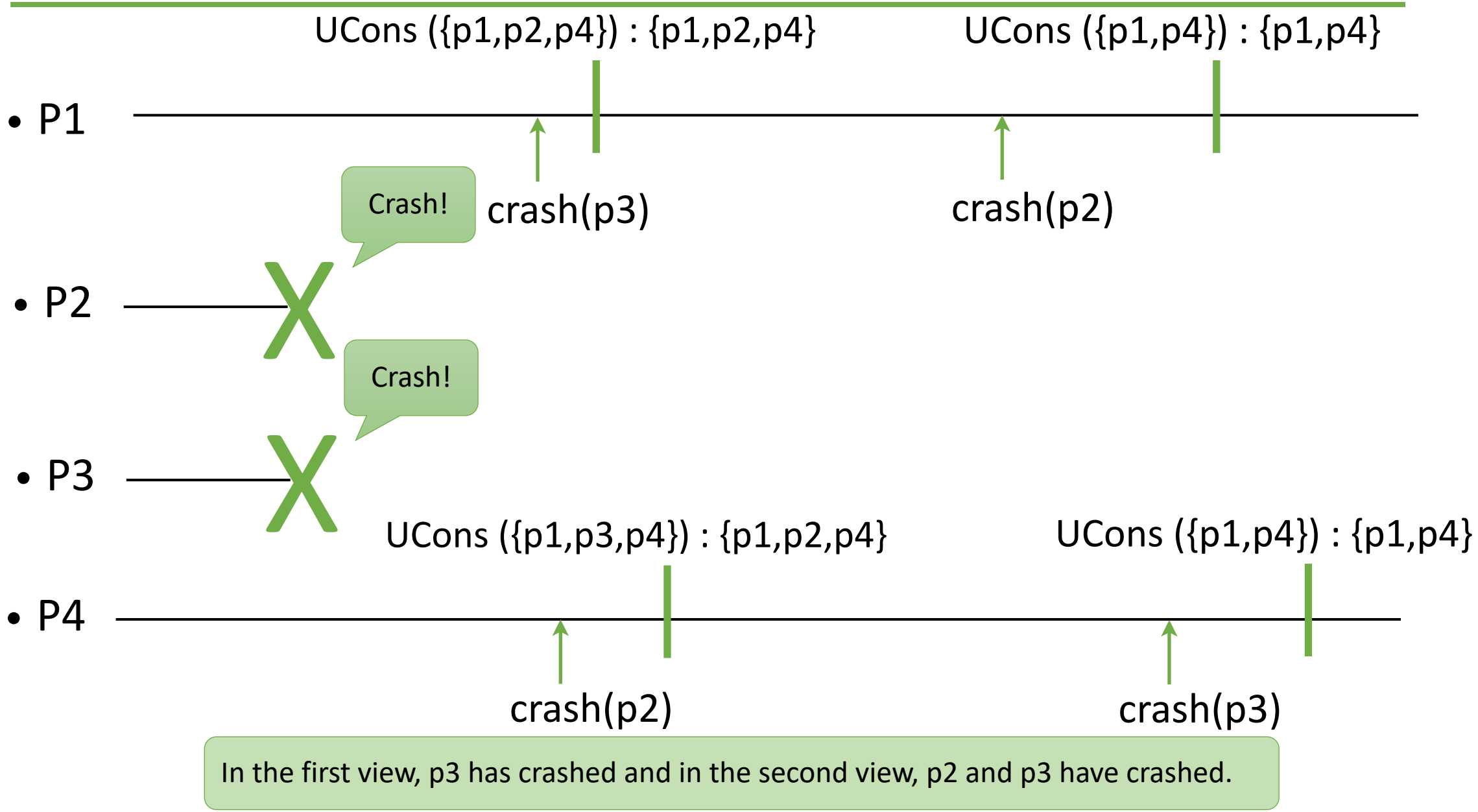
upon event < uc[i], decide(M') > and i = id **do**

M := M'

wait := false

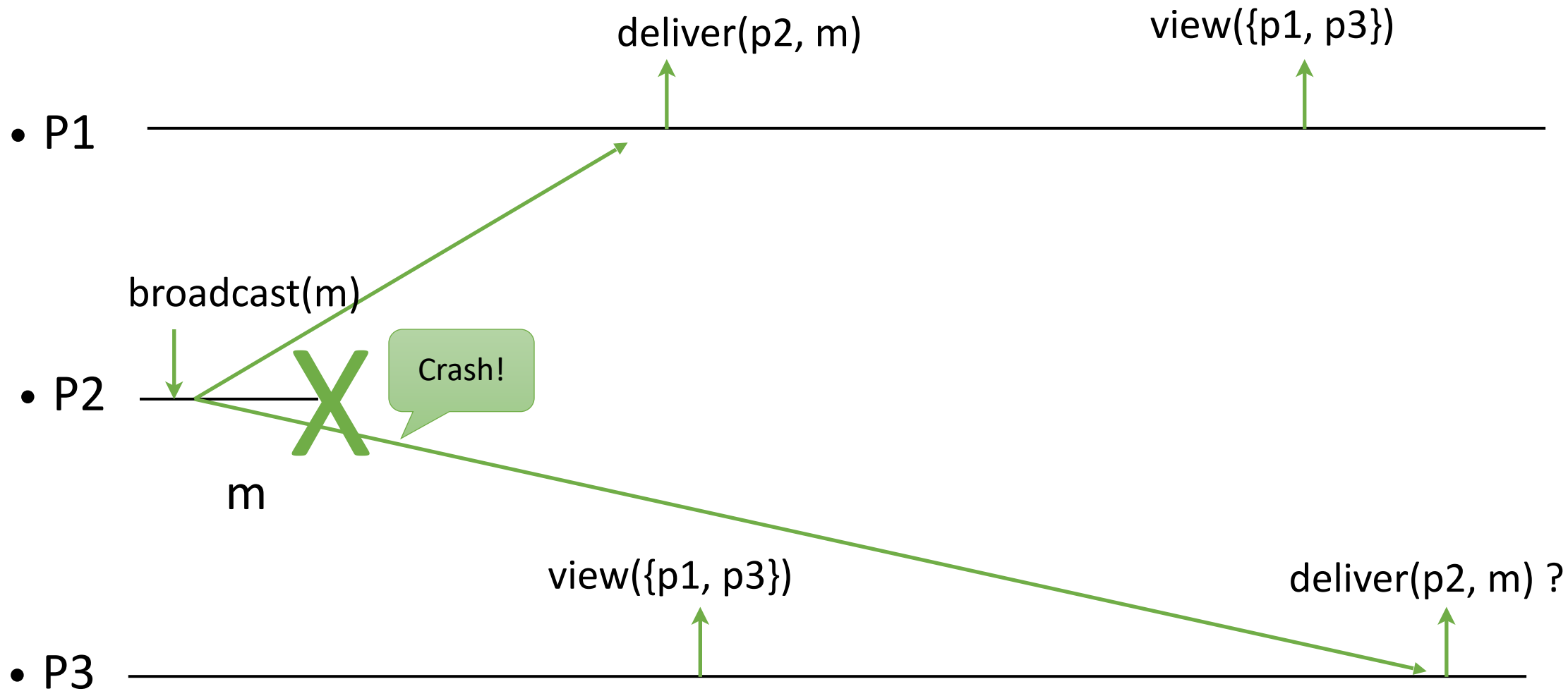
trigger < view (id, M) >

GM Algorithm



In the first view, p3 has crashed and in the second view, p2 and p3 have crashed.

Group Membership and Broadcast



Because of the differences in views, p1 accepts the messages from p2 but p3 does not.

View Synchronous Communication

- **View synchronous broadcast** is an abstraction that results from the combination of group membership and reliable broadcast.

- **View synchronous broadcast** ensures that the delivery of messages is coordinated with the installation of views.

View Synchronous Communication

Events

Request:

< broadcast(m) >

Indication:

< deliver(src, m) >

< view(V) >

View Synchronous Communication

Besides the properties of

group membership (GM1-GM4) and

reliable broadcast (RB1-RB4),

the following property needs to be ensured:

VS: If a process delivers a message m from process p in view V, then m was broadcast by p in view V.

View Synchronous Communication

- If the application **keeps broadcasting** messages, because of the VS property, the view synchrony abstraction might be **unable to Install** a new view; the abstraction would be impossible to implement.
- We introduce a specific event **block** for the abstraction to block the application from broadcasting messages. The application accepts by **block-OK**. This only happens when another process crashes.

View Synchrony

Events

Request:

< broadcast(m) >

< block-OK >

Indication:

< deliver(src, m) >

< view(V) >

< block >

VSC Protocols

- Protocol 1: TRB-based
- Protocol 2: Consensus-based

TRB-based Algorithm

Idea:

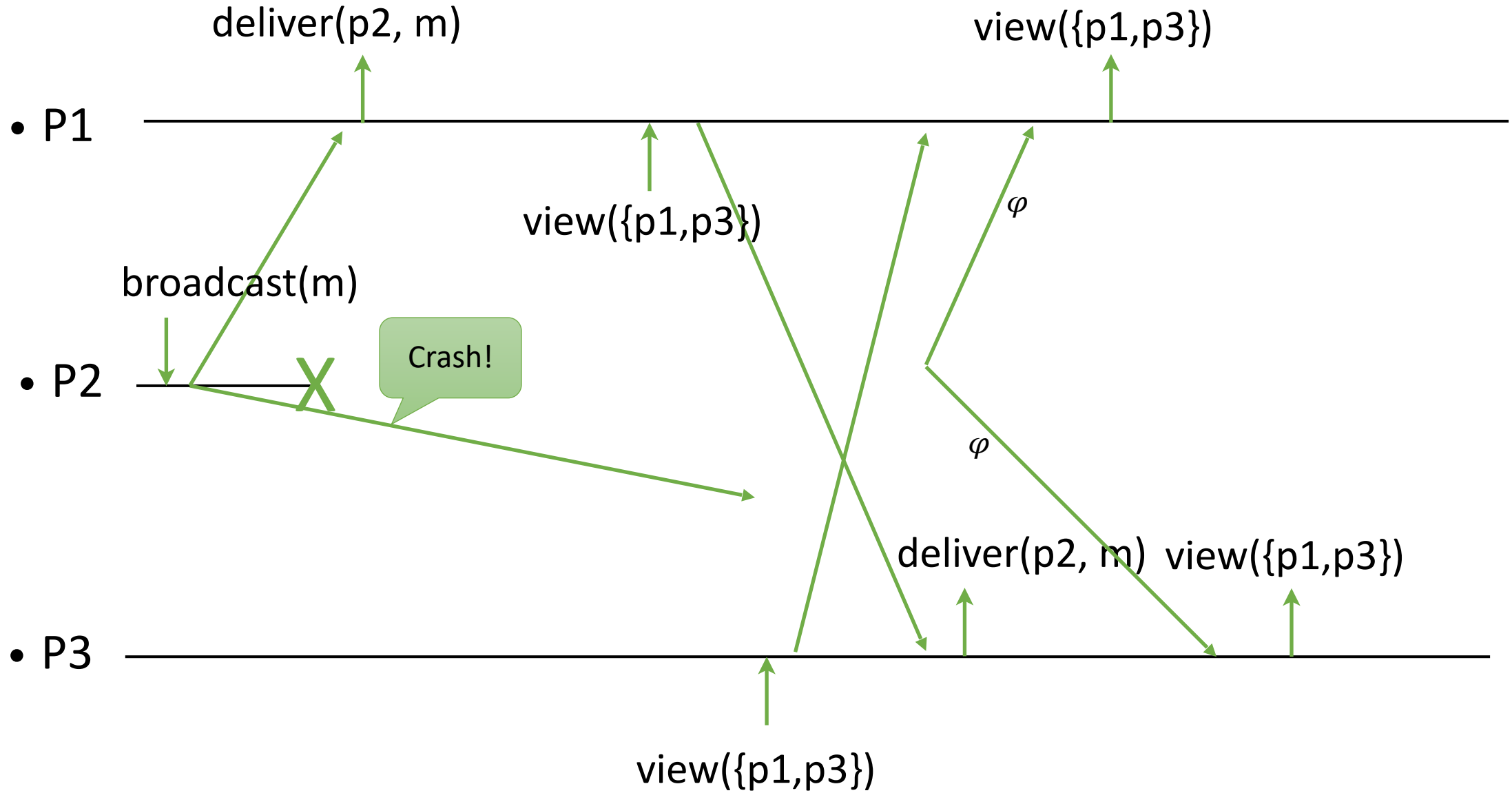
Before delivering a view change, use terminating reliable broadcast (TRB) to communicate delivered messages.

TRB-based VSC Algorithm

Idea:

- When a view change is received, add it to a queue. When there is no other ongoing view change, take a new view from the queue and block broadcasting new messages. Then update other processes with the messages that have been received in the current view. Wait for the update message from every process in the current view. Terminating reliable broadcast (TRB) is used to receive a (dummy) message even if the sender crashes. Then, install the new view and unblock broadcasting of new messages.
- Why TRB? Before moving to the next view, in order to preserve the agreement property of Reliable Broadcast, a correct process needs to deliver messages that other correct processes have delivered in the current view. In order to preserve the completeness property of Group Membership, TRB is used so that processes do not get stuck waiting for messages to arrive from crashed processes.
- When a new message is broadcast, and broadcasting is not blocked, broadcast it using best-effort broadcast (BEB). When it is delivered, deliver the message.

TRB-based VSC Algorithm



TRB-based VSC Protocol

Implements:

ViewSynchronousCommunication (vsc).

Uses:

GroupMembership (gm).

TerminatingReliableBroadcast (trb).

BestEffortBroadcast (beb).

upon event < Init > **do**

(vid, M) := (0, Π)

delivered := ∅

vDelivered := ∅

views-queue := ∅

changing := blocked := false

updated := ∅

vid: the current view id
M: the current member processes
delivered: all the delivered messages
vDelivered: messages delivered in the current view
views-queue: the queue of pending views
changing: if the view is changing
blocked: if broadcasting is blocked
updated: processes whose updates are received

TRB-based VSC Protocol

upon event <broadcast(m)> and (blocked = false) **do**

delivered := delivered ∪ {m}

vDelivered := vDelivered ∪ {(self, m)}

trigger <deliver(self, m)>

trigger <beb, broadcast([vid, m])>

Broadcasting is accepted only if it is not currently blocked.

Although beb delivery performs the first three lines as well, they are needed here so that the message is not lost if a view change is started right after this broadcast event.

TRB-based VSC Protocol

upon event <beb, deliver(src, [v, m])> **do**
    if (vid = v) and (m ∉ delivered) **then**
        delivered := delivered ∪ {m}
        vDelivered := vDelivered ∪ {(src, m)}
        trigger < deliver(src, m) >

The condition vid = v is needed because if a process that is not a member of the current view broadcasts a message, its message is not communicated during the view change. Then, its message can be delivered late to this handler. The late message has to be dropped.
The condition m not in delivered is needed to prevent duplicate delivery at the sender itself.

TRB-based VSC Protocol

upon event < gm, view(V) > **do**
    enqueue V to views-queue

upon (views-queue ≠ ∅) and (changing = false) **do**
    changing := true
    trigger <block>

upon < block-OK > **do**
    blocked := true
    trigger <trb, broadcast([vid, vDelivered])>

TRB-based VSC Protocol

upon <trb, deliver(p, [v, vDel])> **where** v = vid **do**
    updated := updated ∪ {p}
    if (vDel ≠ φ)
        forall (s, m) ∈ vDel and m ∉ delivered **do**
            delivered := delivered ∪ {m}
            trigger <deliver (s, m)>

upon (updated = M ∖ {**self**}) and (blocked = true) **do**
    (vid, M) := dequeue(views-queue)
    changing := blocked := false
    vDelivered := ∅
    updated := ∅
    trigger < view (vid, M) >

We do not need to wait to trb-deliver from self.

The current process self must have received the view change event and blocked new broadcast requests.

Consensus-Based View Synchrony

Idea:

GM and TRB both use consensus.

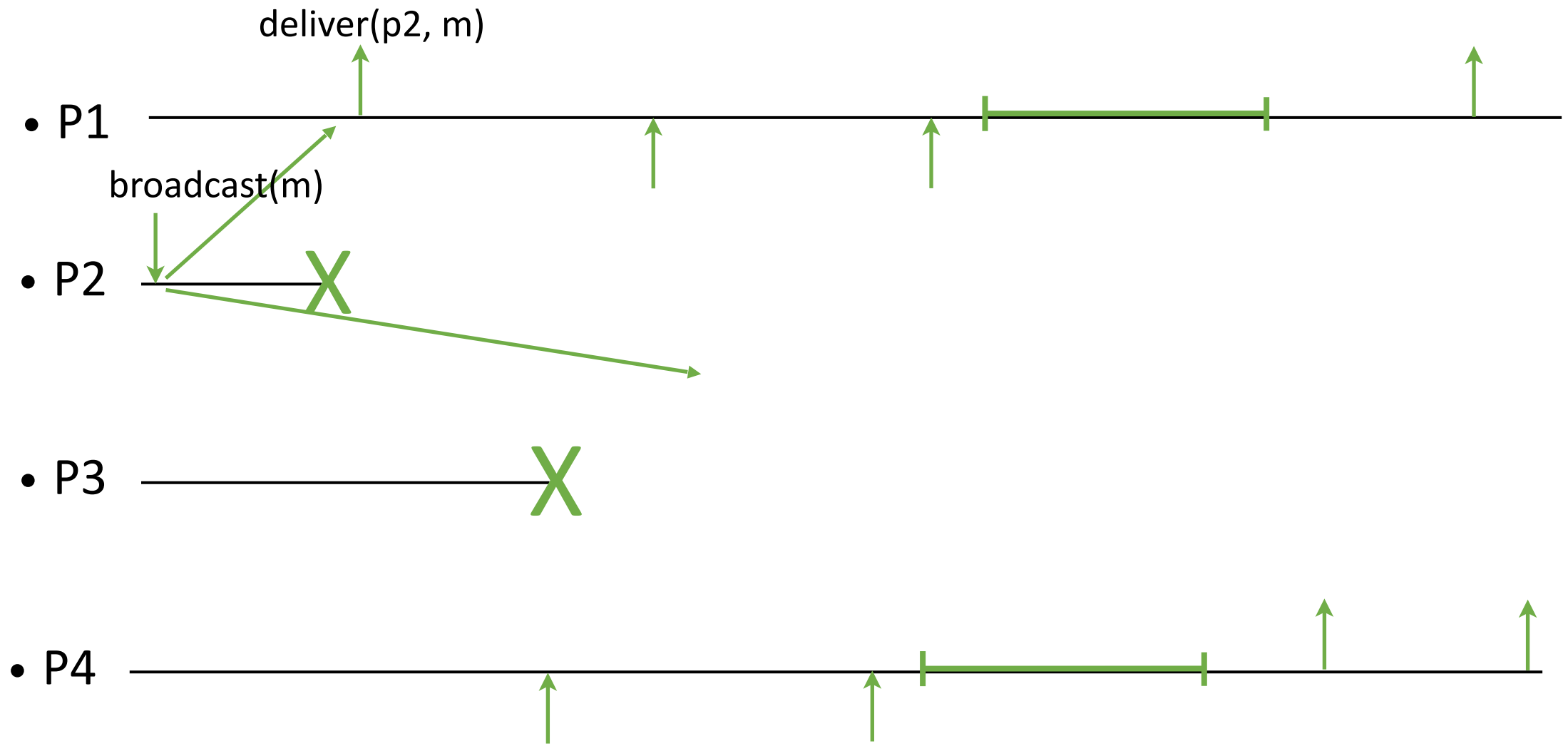
Directly use rounds of consensus to agree on the set of processes and delivered messages.

Consensus-Based View Synchrony

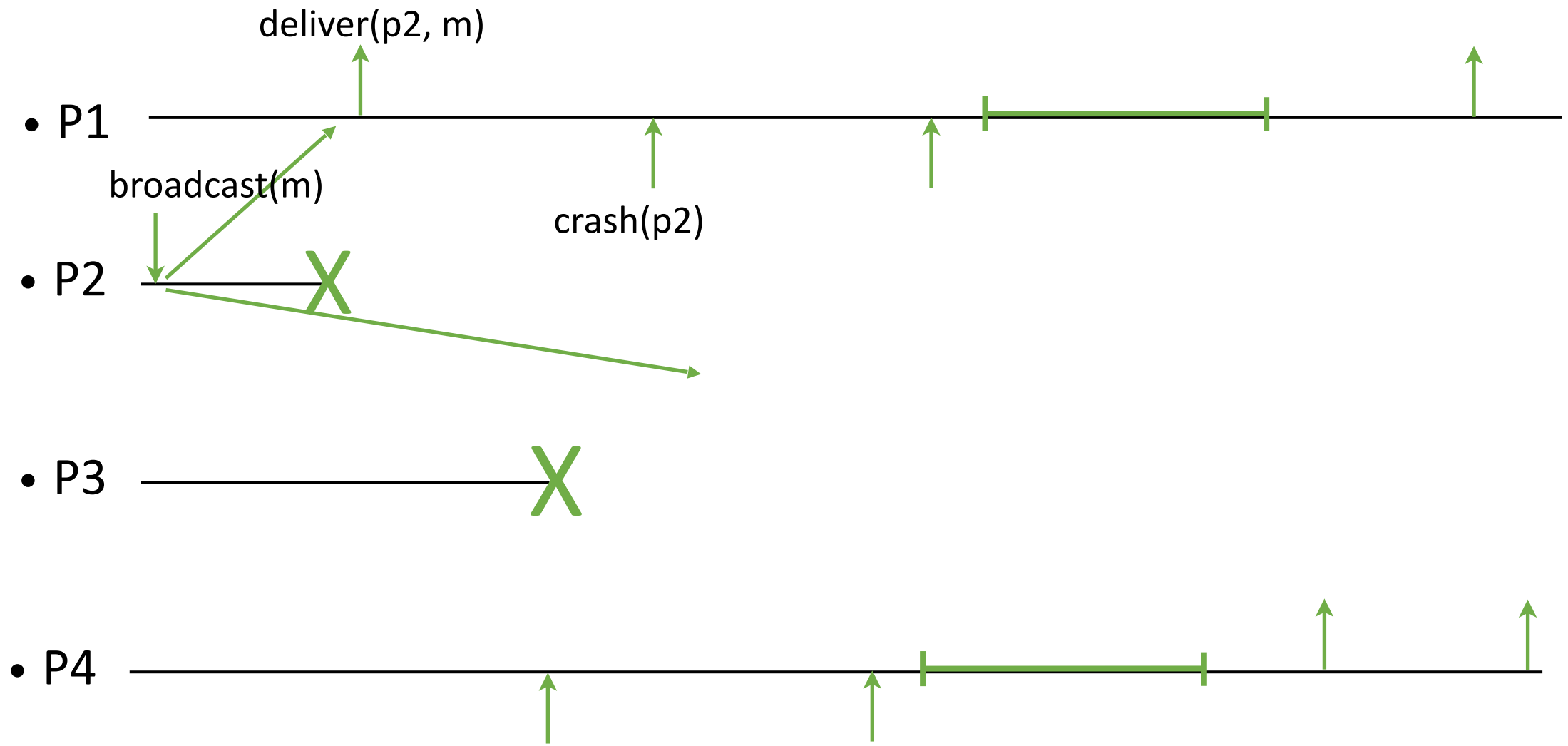
Idea:

- When a process detects a failure, it broadcasts the messages that it has delivered in that round and waits. Once the update messages from all correct processes are delivered, a consensus instance is used to agree on the correct set of processes and the accompanying set of messages from each process.
- The update messages arrive from correct processes. Missing the messages delivered at the incorrect processes does not violate the agreement property of the broadcast. This is not uniform agreement.
- Instead of launching a group membership plus parallel instances of TRBs, we use one consensus instance and parallel broadcasts for every view change.

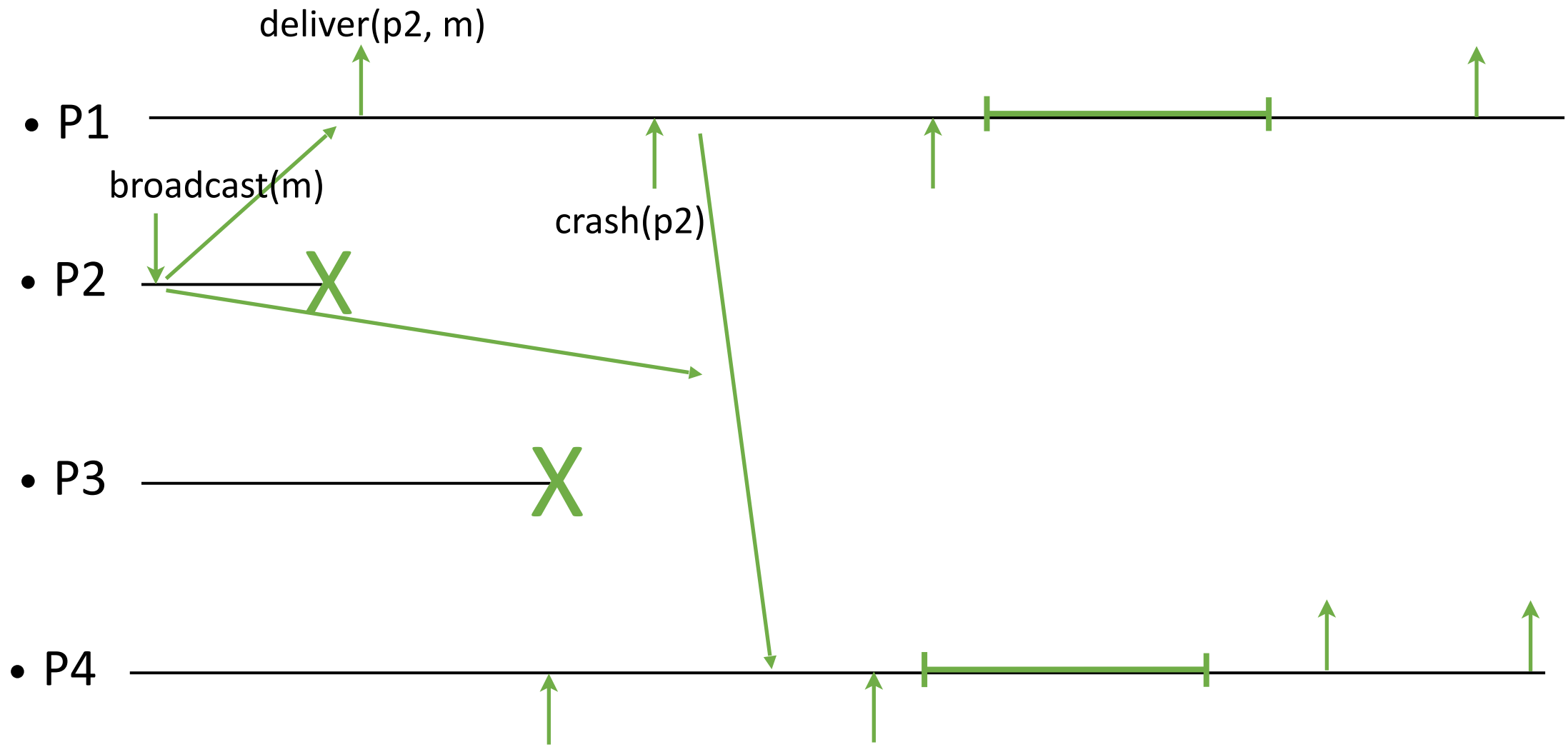
Consensus-Based VSC Algorithm



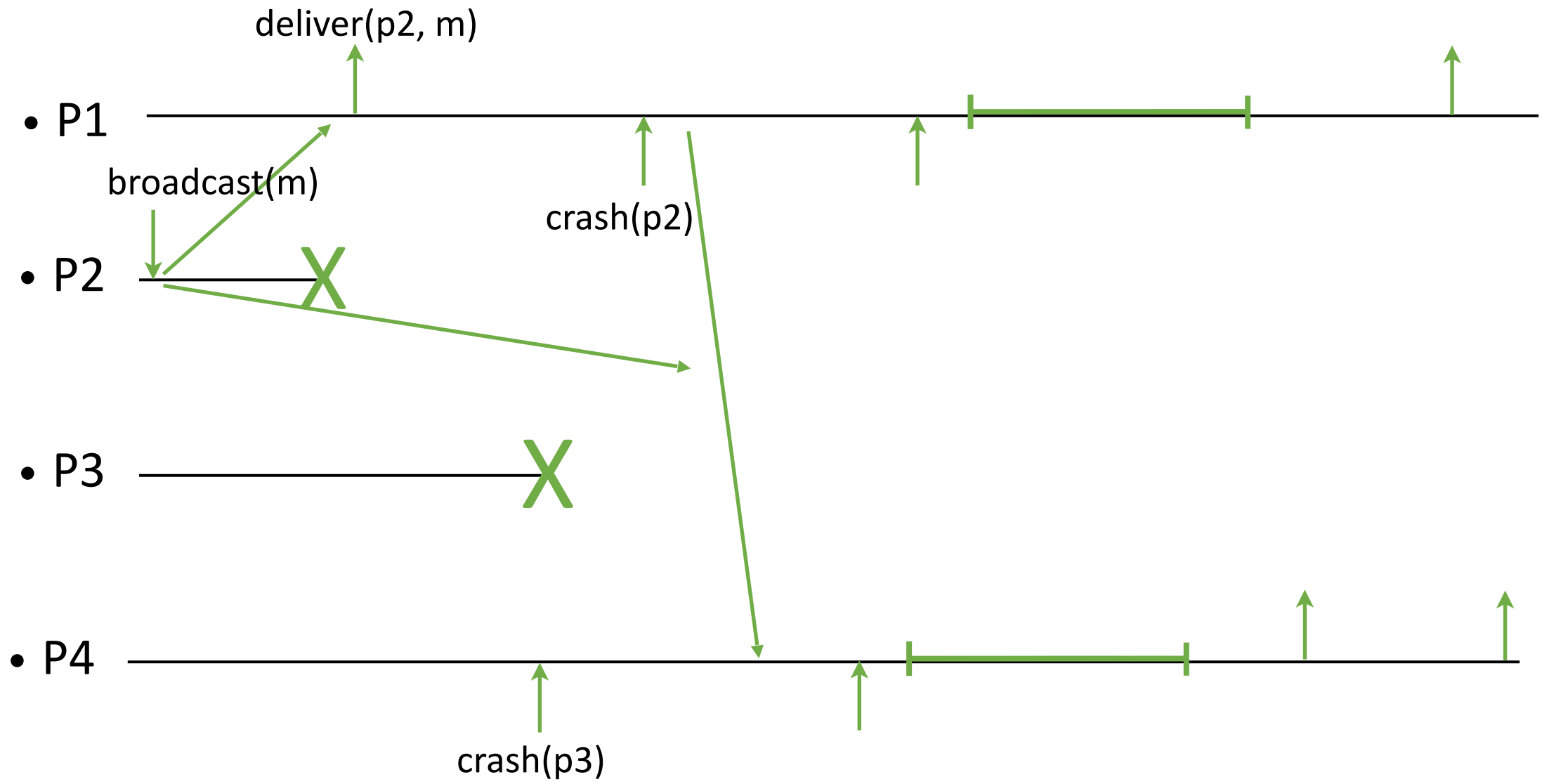
Consensus-Based VSC Algorithm



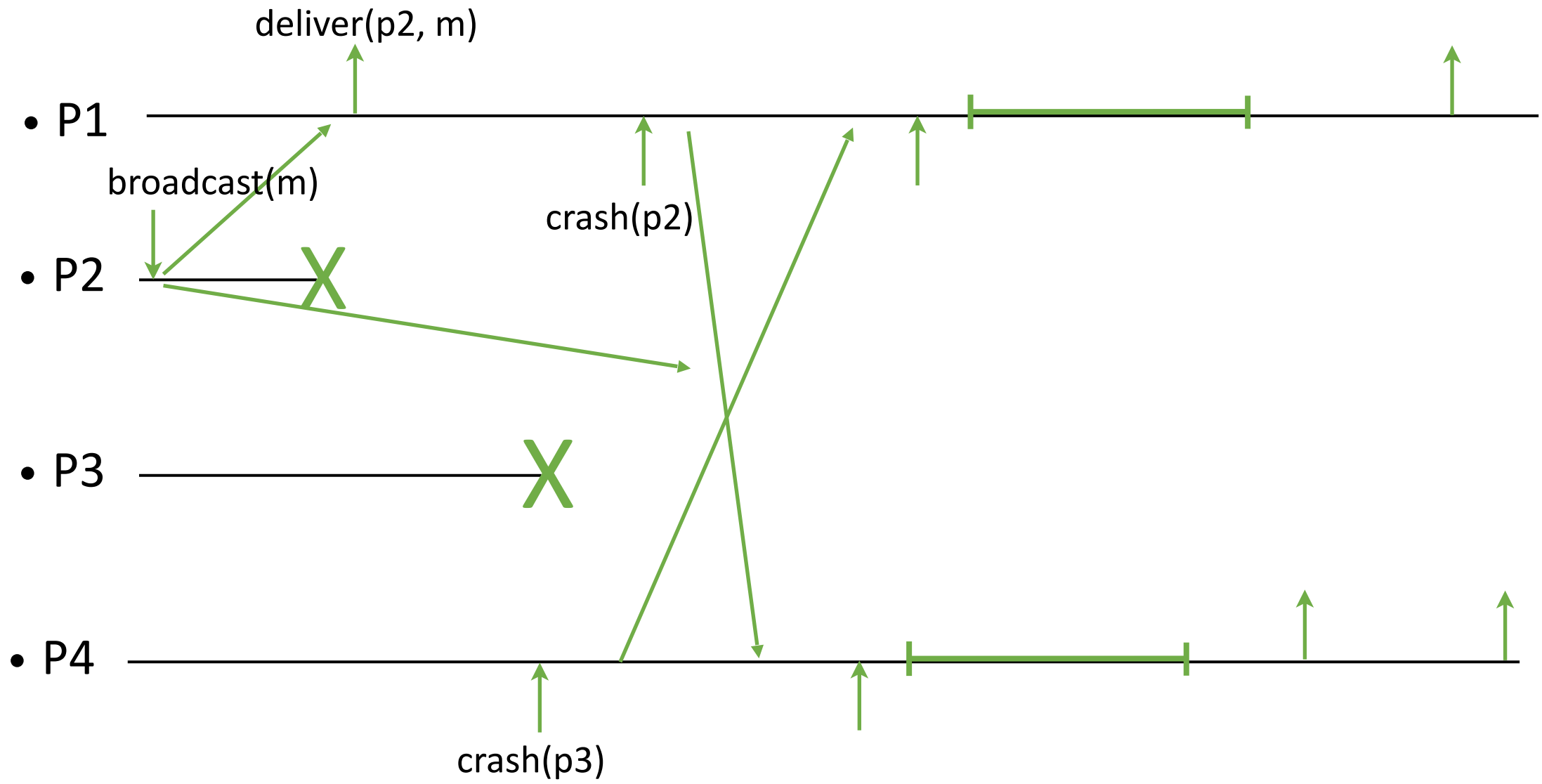
Consensus-Based VSC Algorithm



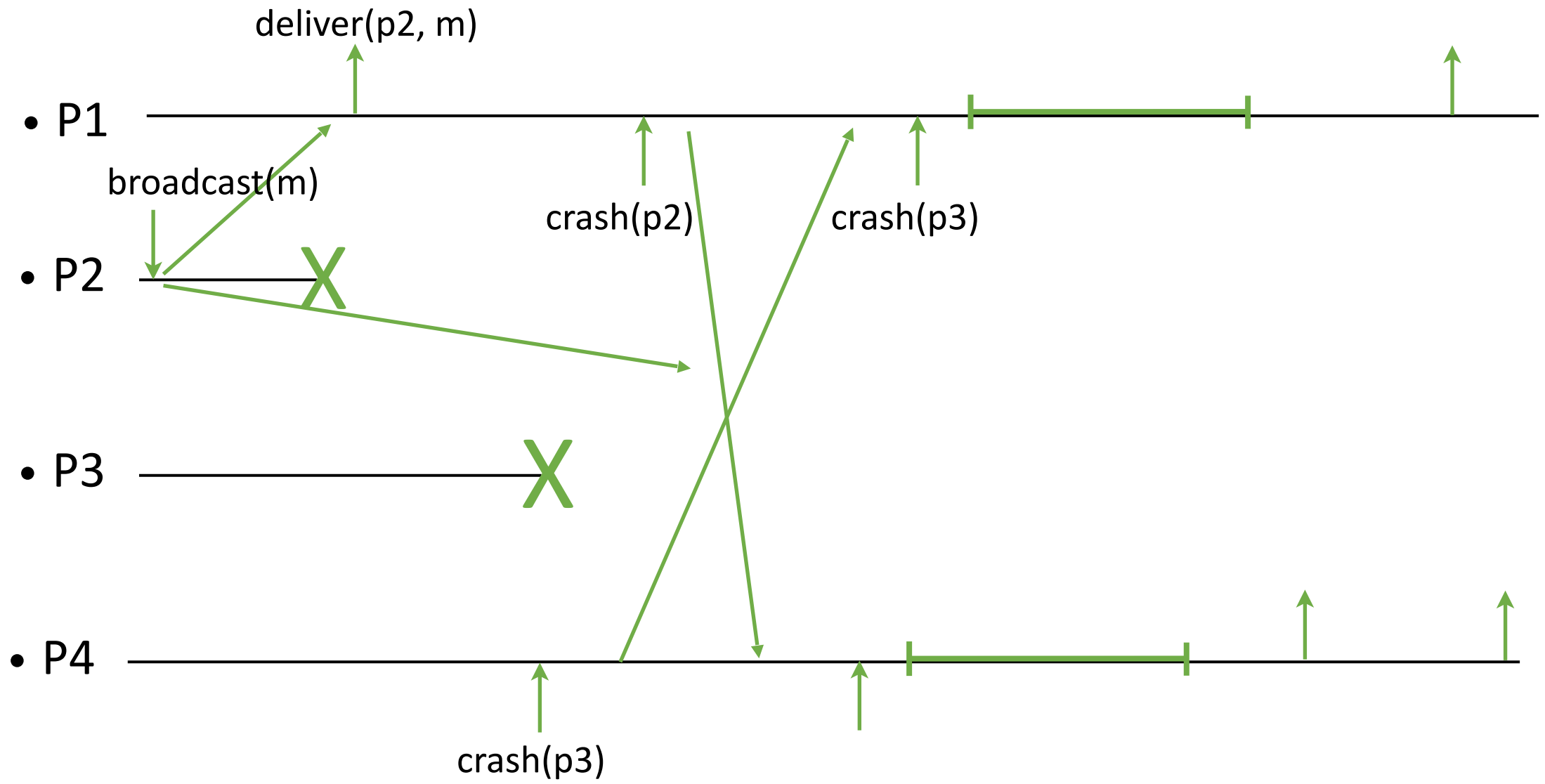
Consensus-Based VSC Algorithm



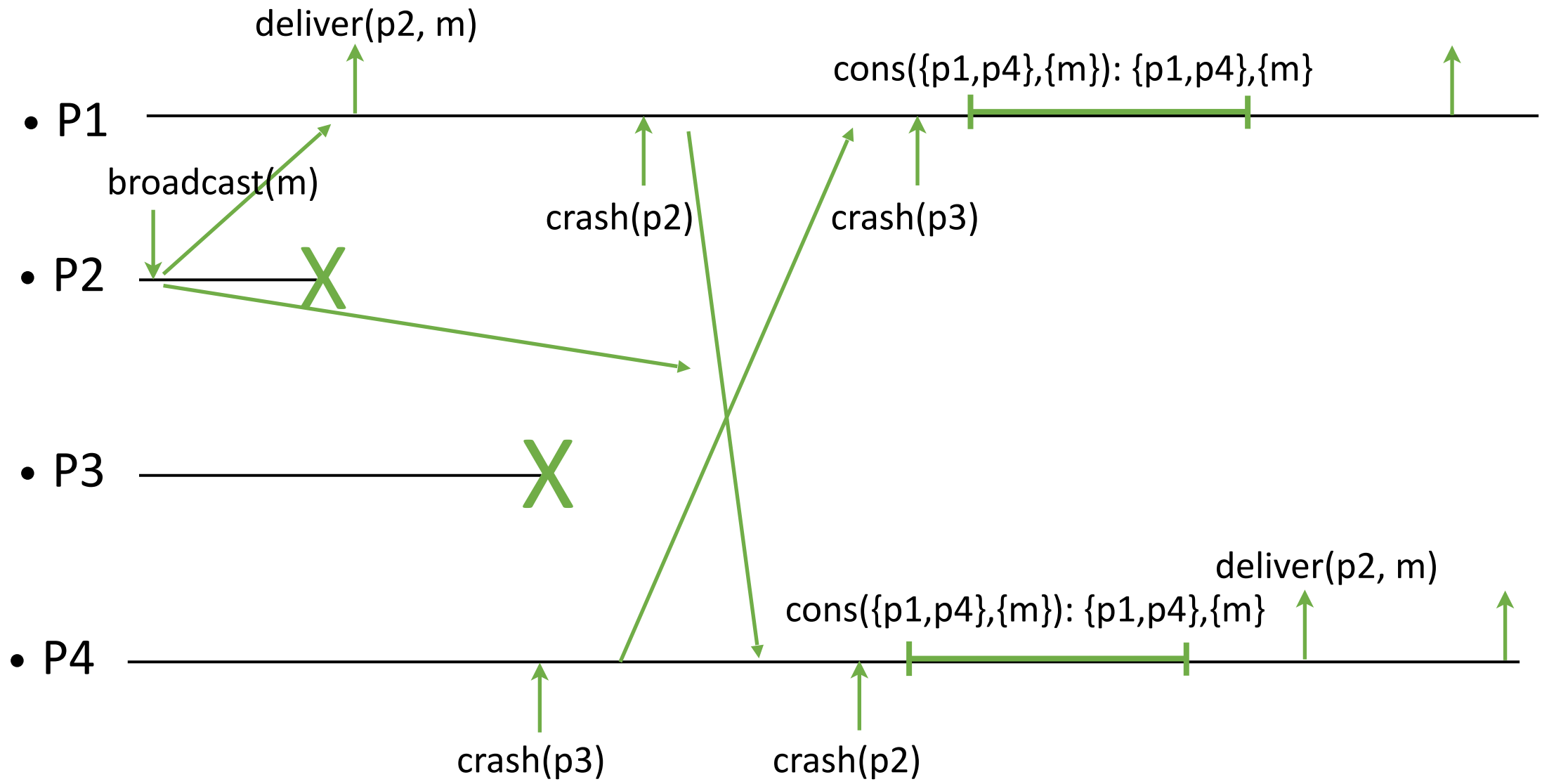
Consensus-Based VSC Algorithm



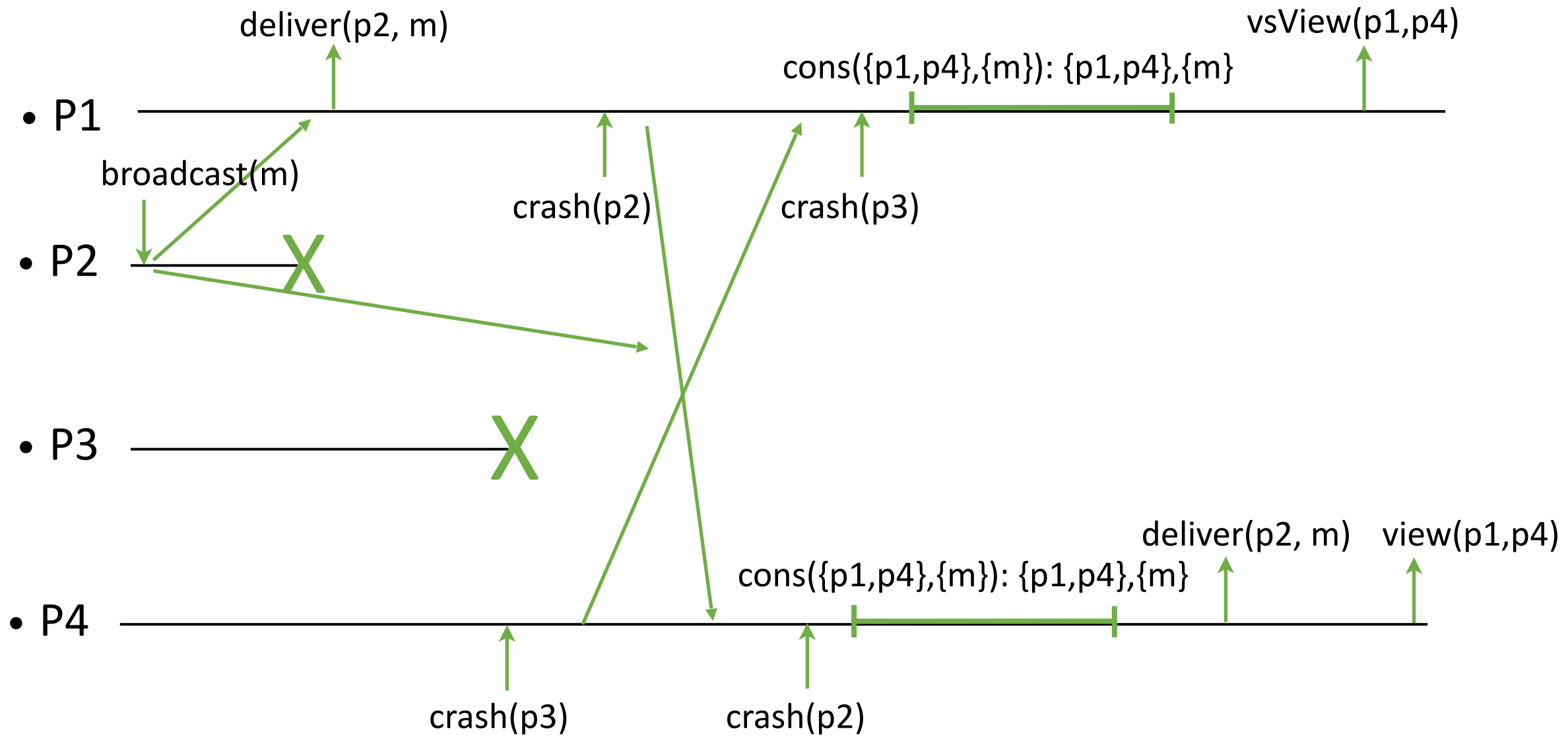
Consensus-Based VSC Algorithm



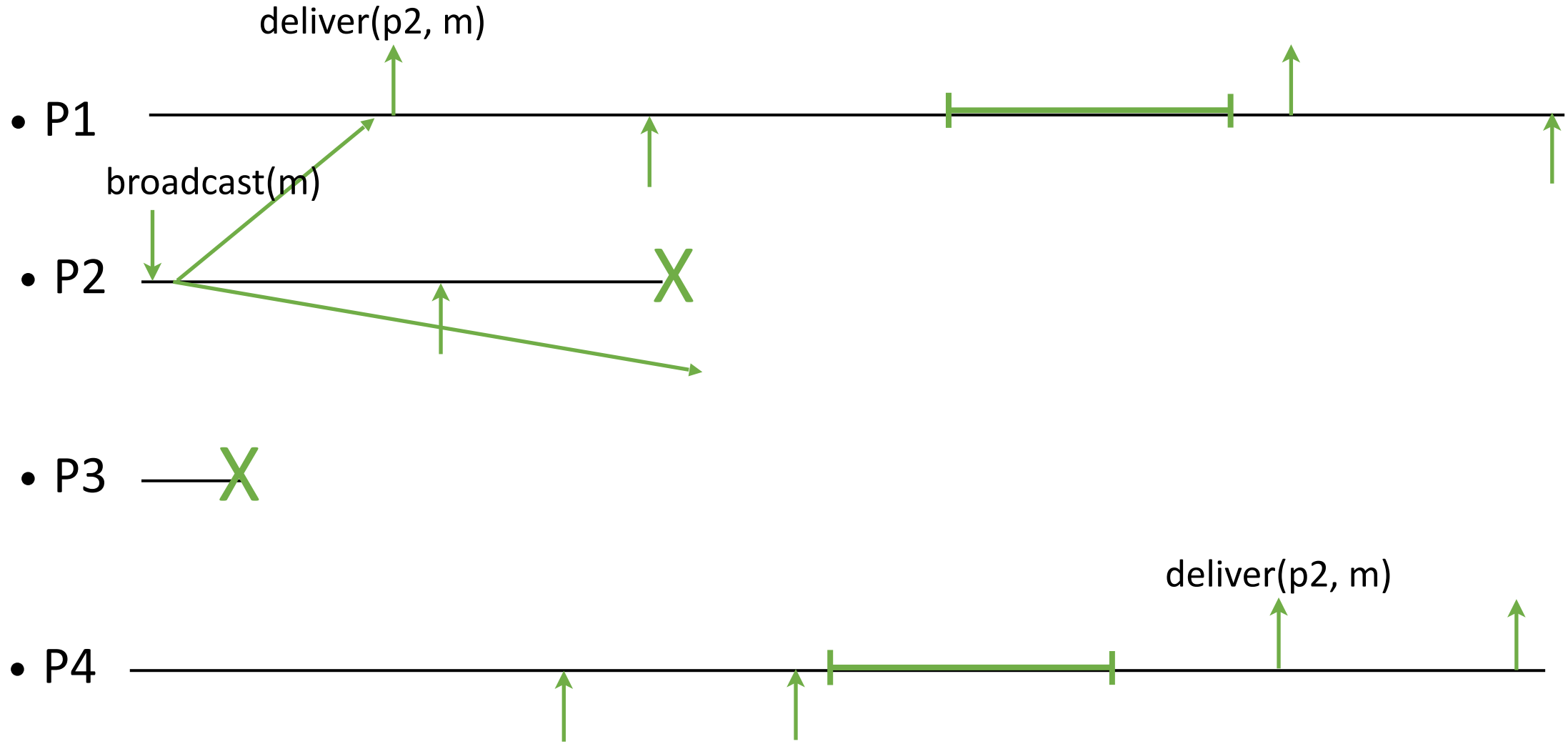
Consensus-Based VSC Algorithm



Consensus-Based VSC Algorithm

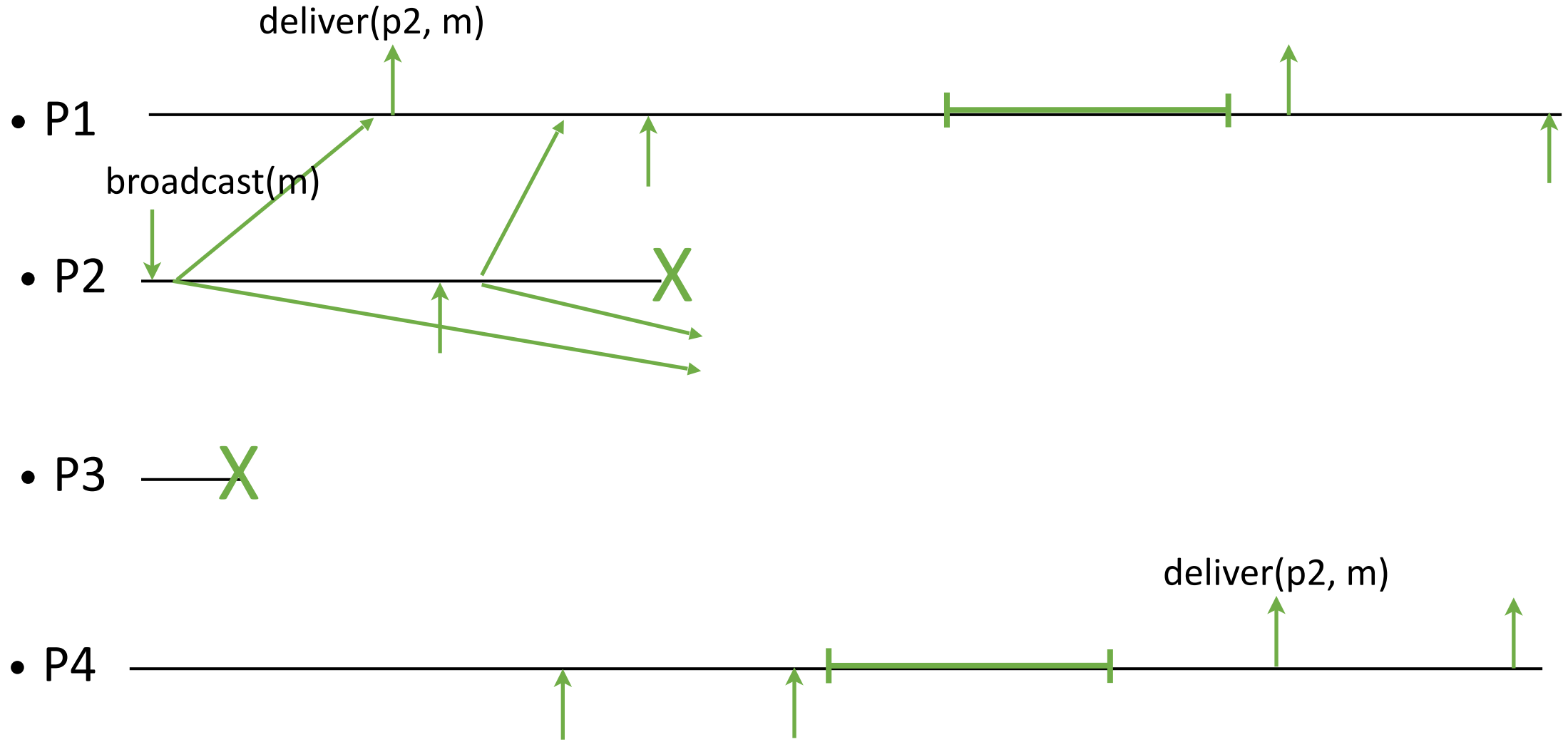


Consensus-Based VSC Algorithm



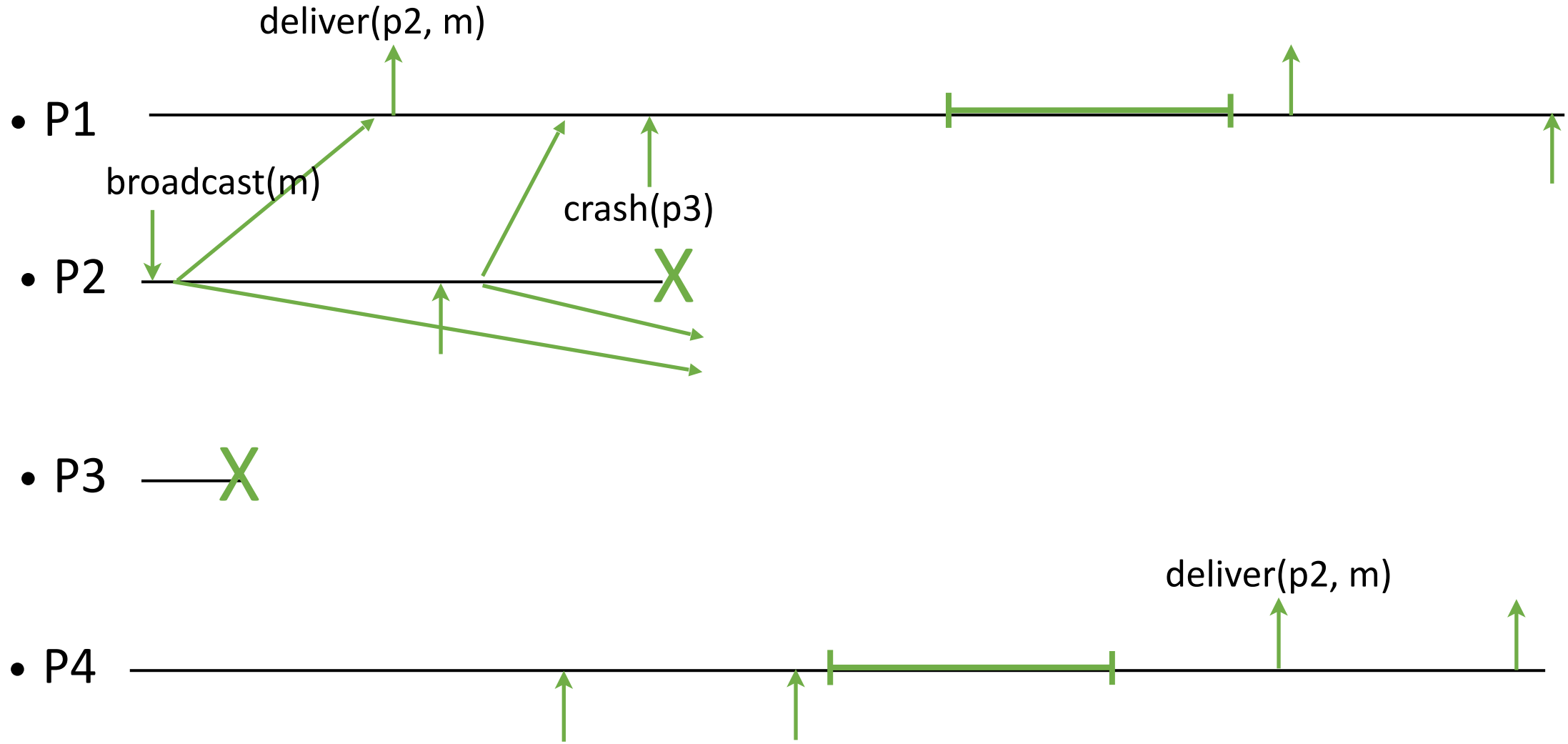
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



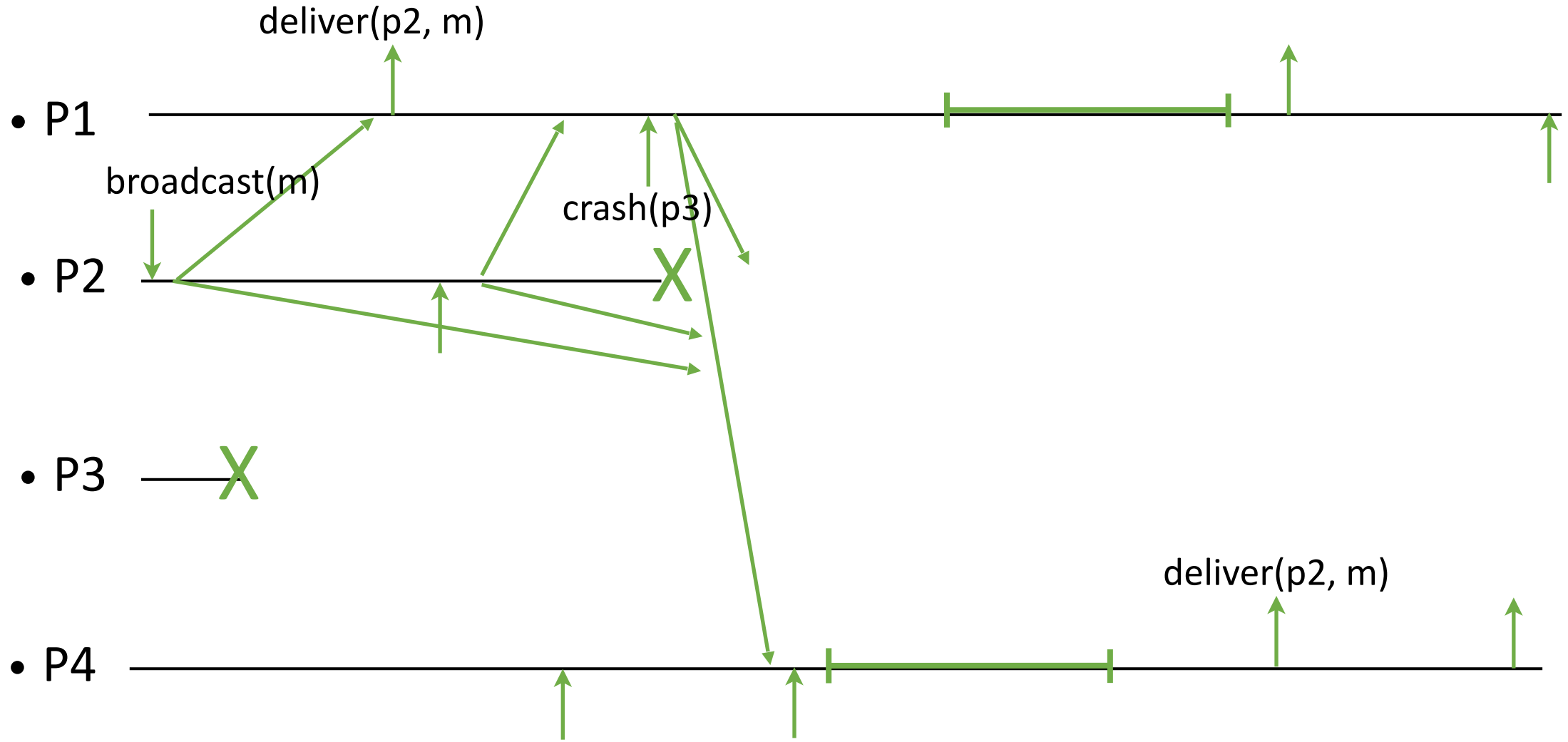
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



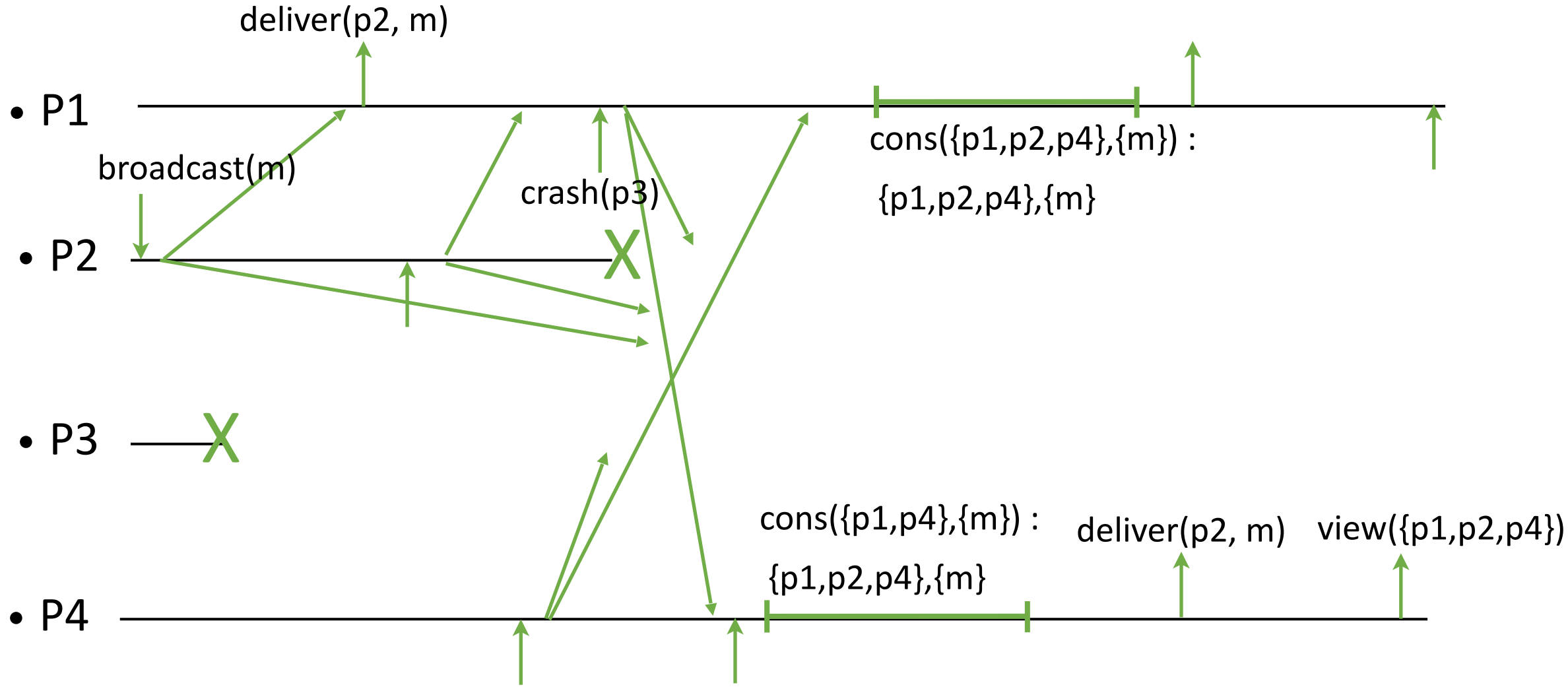
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



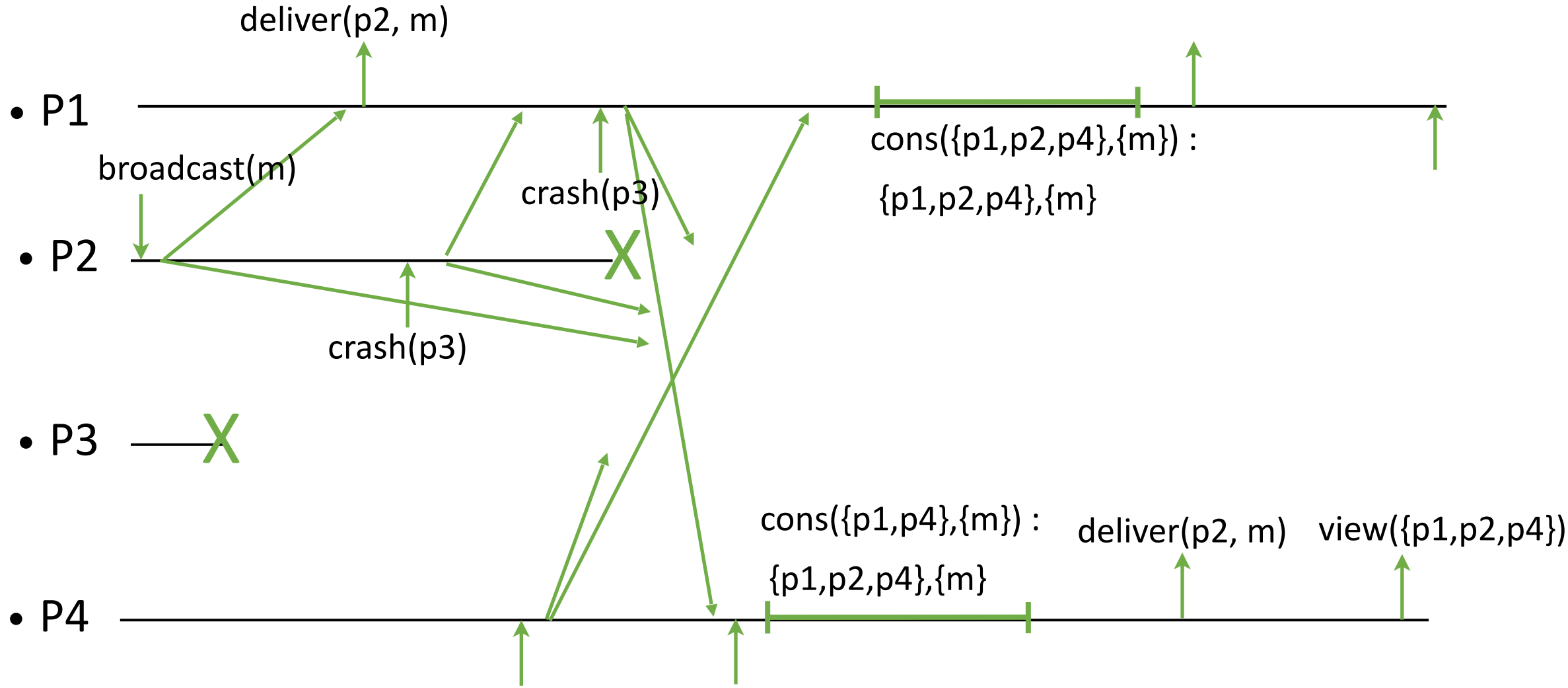
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



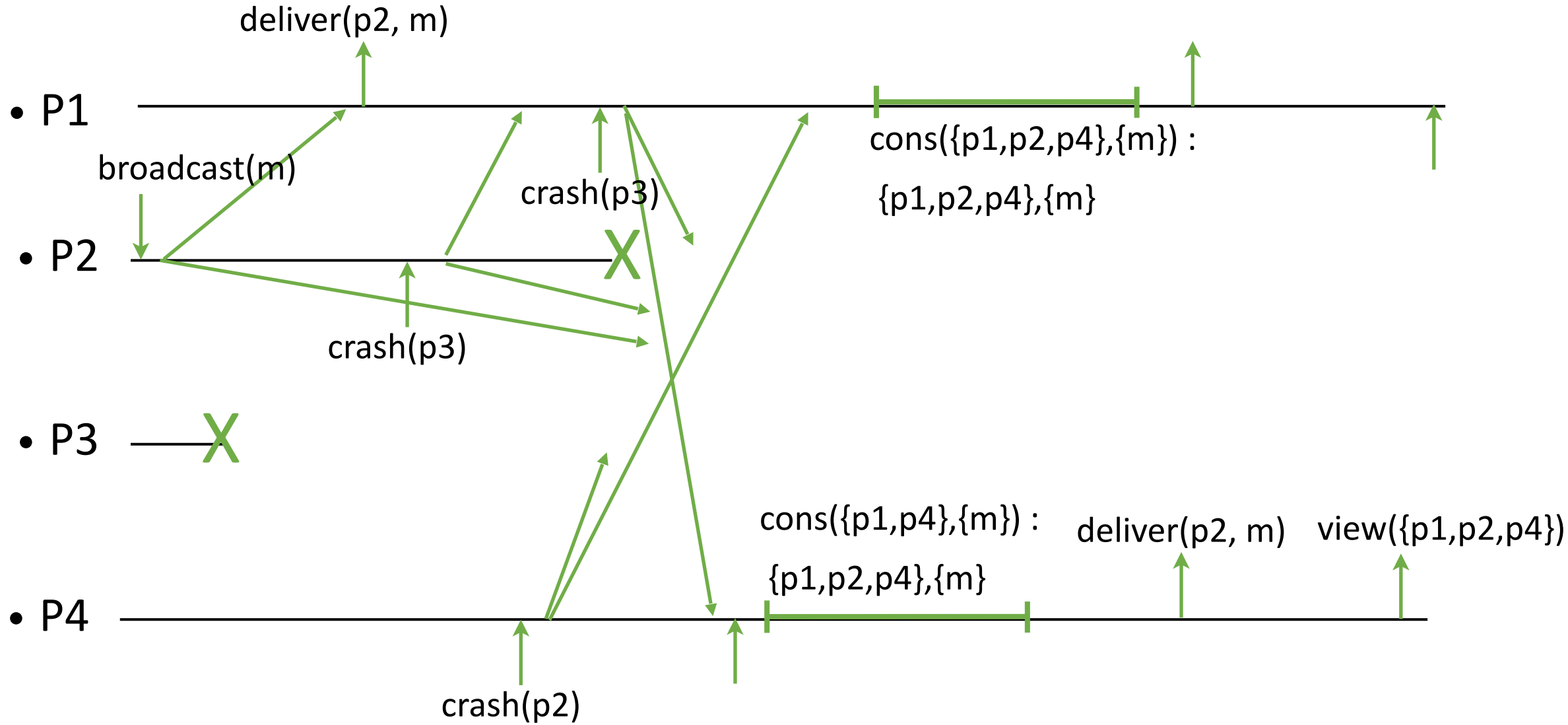
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



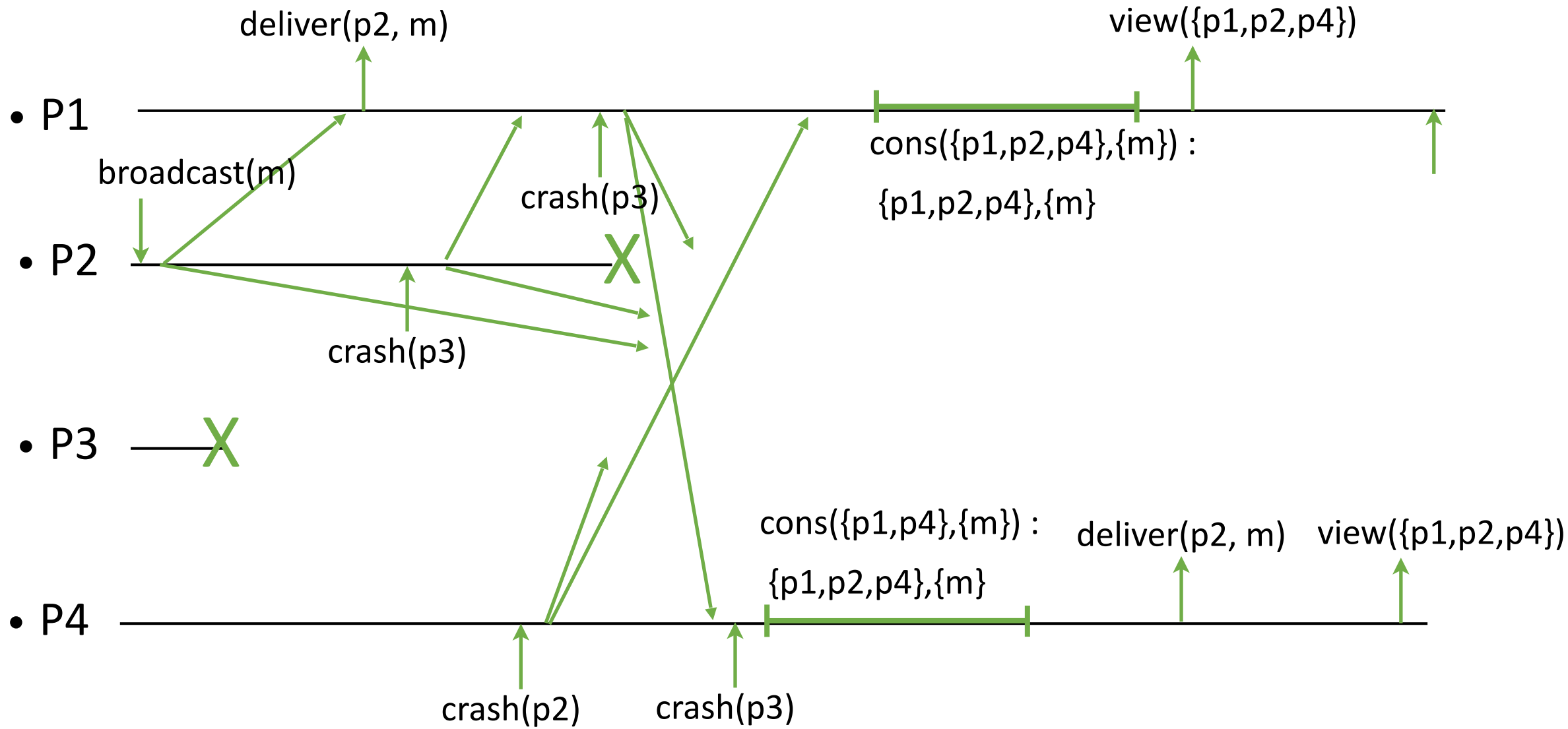
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



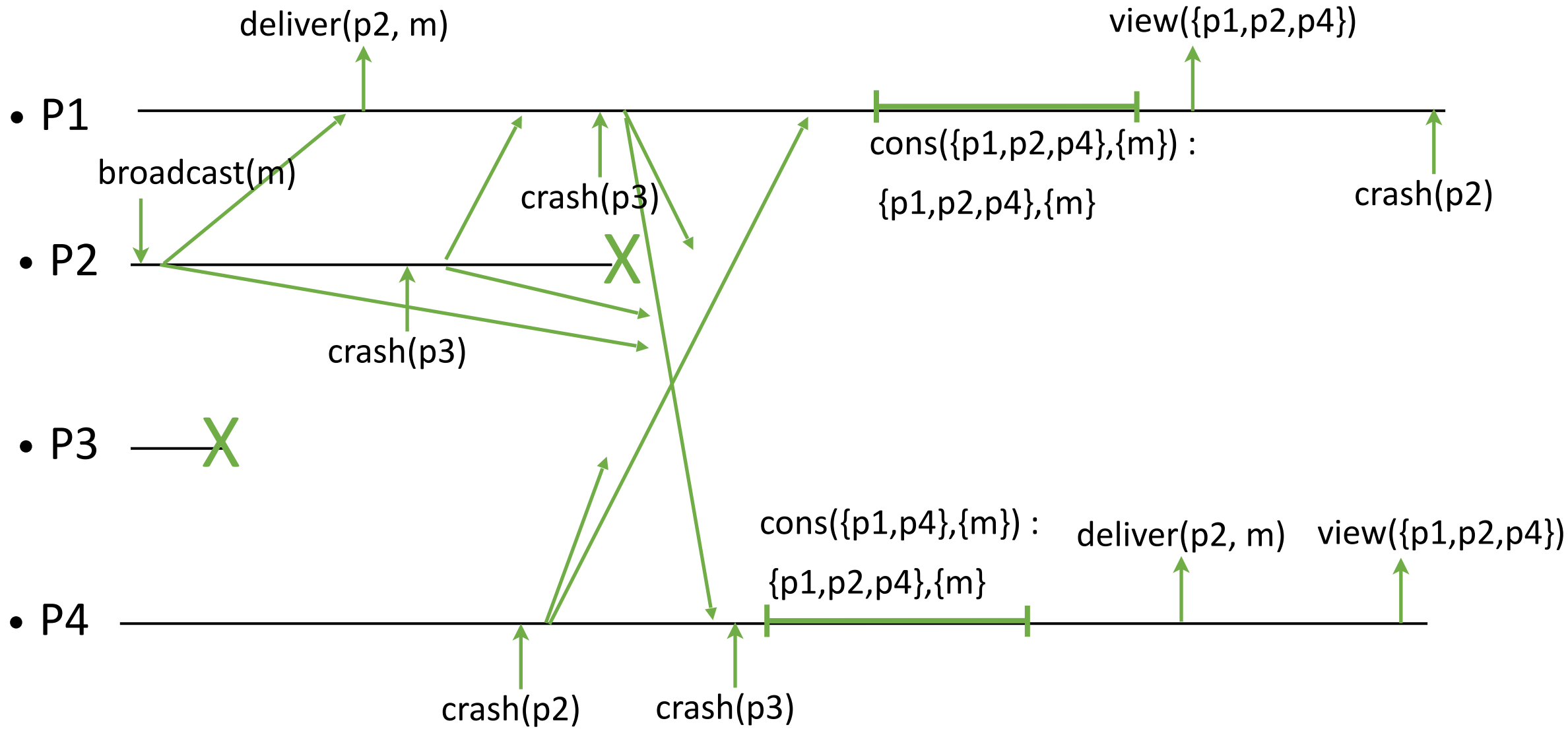
Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm



Process P1 is the only process that has delivered m from P2, and has not heard that it has crashed. The process p1 proposes p2 to be in the next view. He must send m with his proposal, so that others deliver it in the current view.

Consensus-Based VSC Algorithm

Implements: ViewSynchronousCommunication (vsc).

Uses:

UniformConsensus (uc) a sequence.

BestEffortBroadcast (beb).

PerfectFailureDetector (P).

upon event < Init > **do**

view := (0, Π)

correct := Π

changing := blocked := false

delivered := seen := ∅

changing: if the view is changing
blocked: if broadcasting is blocked
delivered: all the delivered messages in this view
seen: mapping from processes to messages delivered from them

Consensus-Based VSC Algorithm

upon event < broadcast(m) > and (blocked = false) **do**

delivered := delivered ∪ {m}

trigger < deliver (self, m) >

trigger < beb, broadcast([vid, m]) >

upon event <beb, deliver(src, [v, m])>

where v = vid and m ∉ delivered and blocked = false **do**

delivered := delivered ∪ {m}

trigger <deliver (src, m)>

Consensus-Based VSC Algorithm

upon event <P, crash (p)> **do**
    correct := correct \ {p}
    if correct ⊊ M and changing = false **then**
        changing := true
        trigger < block >

upon < block-OK > **do**
    blocked := true
    trigger <beb, broadcast(Seen[vid, delivered])>

upon <beb, deliver(src, Seen[v, del])> **where** v = vid **do**
    seen[src] := del
    if forall p ∈ correct, seen[p]≠⊥ **then**
        vid := vid + 1
        initialize uc[vid]
        trigger < uc[vid], propose([correct, seen]) >

Consensus-Based VSC Algorithm

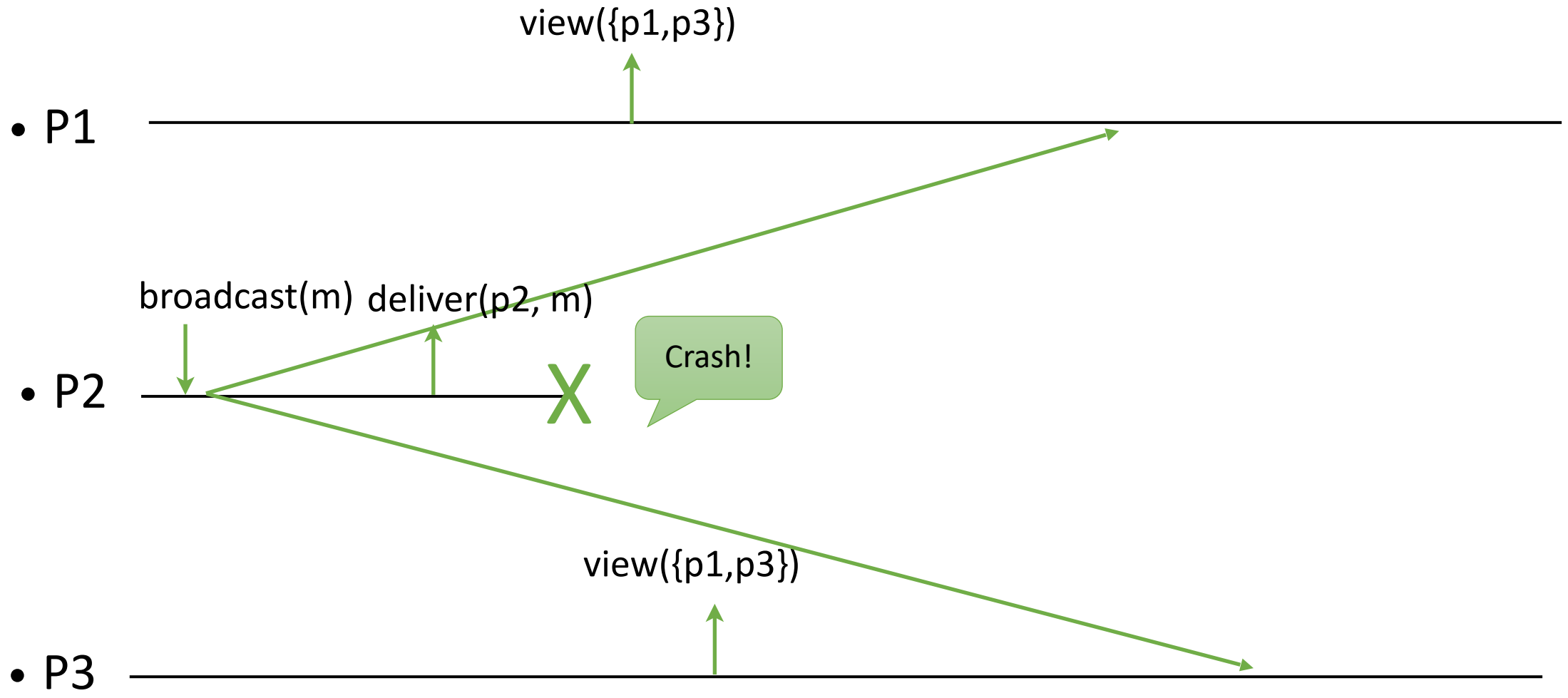
upon <uc[v], decide([M', S])> where v = vid **do**
    forall p ∈ M', (s, m) ∈ S[p] such that m ∉ delivered **do**
        delivered := delivered ∪ {m}
        trigger <deliver(s, m)>
    M := M'
    changing := blocked := false
    seen := delivered := ∅
    trigger < view(vid, M)>

Uniform View Synchrony

We now combine the properties of

- **Group Membership (GM1-GM4)** – which is already uniform. (No two processes (correct or not) install different sets in the same view.)
- **Uniform Reliable Broadcast (RB1-RB4)** – which we require to be uniform
- **VS**: If a process delivers a message m from process p in view V, then m was broadcast by p in view V.

Uniformity?



The message m is delivered in P2 but is not delivered in P1 and P3.

Uniform View Synchrony

Using uniform reliable broadcast instead of best effort broadcast in the previous algorithms does not ensure the uniformity of the message delivery.

Uniformity

Idea:

- Deliver a message only when all the correct processes acknowledge receiving it.
- This is similar to uniform reliable broadcast.

Uniform VSC Algorithm

upon event < Init > **do**

(vid, M) := (0, S)

correct := S

changing := blocked := false

pending := delivered := seen := ∅

for all m: ack(m) := ∅

Uniform VSC Algorithm

upon event < broadcast(m) > and (blocked = false) **do**

pending := pending ∪ {(self, m)}

trigger <beb, broadcast([vid, self, m]>

upon event <beb, deliver(src, [id, s, m])>

where id = vid and blocked = false **do**

ack(m) := ack(m) ∪ {src}

if (s, m) ∉ pending **then**

pending := pending ∪ {(s, m)}

trigger <beb, broadcast([vid, m])>

upon event (s, m) ∈ pending and M ⊆ ack(m) and (m ∉ delivered) **do**

delivered := delivered ∪ {m}

trigger < deliver(s, m) >

Uniform VSC Algorithm

upon event < crash, p > **do**
    correct := correct \ { p }
    if changing = false **then**
        changing := true
        trigger < block >

upon < block-OK > **do**
    blocked := true
    trigger <beb, broadcast(Pending(vid, pending))>

upon <beb, deliver(src, Pending(id, pd))> where id = vid **do**
    seen[src] := pd
    if forall p ∈ correct, seen[src] ≠ ⊥ **then**
        vid := vid + 1
        initialize uc[vid]
        trigger <uc[vid], propose([correct, seen])>

It could send delivered instead of pending, but pending is a superset of delivered. When safe, we want to deliver more. The consensus guarantees that the same set is decided and delivered everywhere.

Uniform VSC Algorithm

upon <uc[id], decide([M', S])> where id = vid **do**
    forall p ∈ M', (s, m) ∈ S[p] such that m ∉ delivered **do**
        delivered := delivered ∪ {m}
        trigger <deliver(s, m)>
    changing := blocked := false
    seen := delivered := pending := ∅
    for all m: ack(m) := ∅
    M := M'
    trigger <view (vid, M)>

Original slides adopted from R. Guerraoui